



Computación Paralela y Distribuida

Prof. Johansell Villalobos Cubillo

Proyecto Final - Entrega I

Integrantes:

Ana Maria Ramírez

María José Miranda

Siloé Campos

Julio Gutiérrez

David Mora

San José, Julio 2026

Introducción

El crecimiento acelerado del comercio electrónico ha incrementado de manera significativa el volumen de datos generados por las interacciones entre usuarios y productos. Cada visualización, adición al carrito y compra constituye una fuente de información valiosa que permite comprender las preferencias de los clientes y ofrecer experiencias de compra más personalizadas. Sin embargo, el procesamiento de millones de estos registros representa un desafío importante en términos de almacenamiento, análisis y capacidad de respuesta de los sistemas tradicionales, lo que hace necesario recurrir a técnicas de computación paralela y procesamiento distribuido.

En este contexto, los sistemas de recomendación se han convertido en un componente imprescindible de las plataformas de comercio electrónico, ya que permiten sugerir productos relevantes para cada usuario a partir de su comportamiento histórico y de los patrones observados en sus interacciones. Estas recomendaciones contribuyen a mejorar la experiencia del usuario, incrementan las tasas de conversión y fortalecen la fidelización de los clientes. No obstante, para que estos sistemas sean efectivos deben ser capaces de procesar grandes volúmenes de datos de manera eficiente y generar recomendaciones en tiempos de respuesta reducidos, incluso ante picos de tráfico como los que ocurren en eventos masivos de ventas (por ejemplo, el Black Friday).

Frente a esta necesidad, el presente proyecto propone el diseño, la implementación y la evaluación de un sistema de recomendación personalizado para comercio electrónico, cuya optimización se logra mediante el aprovechamiento conjunto de paralelismo multinúcleo, aceleración por GPU y procesamiento distribuido. El objetivo es que el equipo comprenda cómo conceptos fundamentales de la computación paralela; como el speedup, la eficiencia, la sincronización y el profiling, se traducen en aplicaciones prácticas que impactan la vida diaria de millones de personas. Concretamente, se busca construir un sistema capaz de analizar en segundos miles de interacciones de usuarios, generar recomendaciones en tiempo real, escalar horizontalmente ante picos de demanda y demostrar cómo CPU, GPU y clústeres distribuidos trabajan de manera articulada en un servicio de uso cotidiano.

Como fuente de datos se utilizará el dataset RetailRocket E-commerce Dataset, el cual contiene aproximadamente 2.7 millones de eventos de interacción entre usuarios y productos,

incluyendo visualizaciones, adiciones al carrito y transacciones, además de información sobre categorías y propiedades de los productos. La magnitud de este conjunto de datos ofrece un escenario adecuado para evaluar el impacto de distintas estrategias de paralelización tanto en tareas de análisis exploratorio y estadístico como en procesos de aprendizaje automático, permitiendo así medir de forma tangible los beneficios de la computación paralela y distribuida aplicada a un problema real de alto impacto: la optimización de un sistema de recomendaciones personalizadas para compras en línea.

Trabajo Relacionado

Los sistemas de recomendación se han convertido en una parte esencial de las plataformas digitales modernas, especialmente en el comercio electrónico. Su función principal es ayudar al usuario a encontrar productos relevantes dentro de catálogos cada vez más amplios. En este tipo de entornos, cada visualización, búsqueda, adición al carrito o compra deja una señal de comportamiento que puede utilizarse para comprender mejor los intereses de los clientes y generar recomendaciones personalizadas.

A lo largo del tiempo se han desarrollado distintos enfoques para construir sistemas de recomendación. Uno de los más utilizados es el filtrado colaborativo, el cual se basa en la relación entre usuarios y productos. La idea central es que, si varios usuarios han mostrado comportamientos similares en el pasado, es probable que compartan intereses hacia ciertos productos. Este enfoque resulta especialmente útil cuando se cuenta con grandes cantidades de interacciones históricas, como ocurre en plataformas de e-commerce.

Dentro del filtrado colaborativo, una de las técnicas más conocidas es la factorización matricial. Este método representa a usuarios y productos mediante vectores de características latentes, lo que permite estimar afinidades entre ambos. En lugar de trabajar directamente con una matriz muy grande y dispersa de interacciones, el sistema aprende representaciones más compactas que facilitan el cálculo de recomendaciones. Esta idea ha sido ampliamente aplicada en escenarios donde se busca predecir preferencias a partir de datos históricos.

En el comercio electrónico, muchas veces no se cuenta con calificaciones explícitas de los usuarios. Es decir, los clientes no siempre indican si un producto les gustó o no mediante una puntuación directa. En su lugar, el sistema debe interpretar señales indirectas, como visitar un producto, agregar al carrito o realizar una compra. A este tipo de información se le conoce

como retroalimentación implícita. Trabajar con esta clase de datos requiere un tratamiento cuidadoso, ya que no todas las acciones tienen el mismo nivel de importancia; por ejemplo, una compra suele representar una señal más fuerte que una simple visualización.

Además de los métodos tradicionales, en años recientes han ganado relevancia los modelos basados en aprendizaje profundo. Estos enfoques permiten representar usuarios y productos mediante embeddings, es decir, vectores aprendidos automáticamente durante el entrenamiento. Arquitecturas como los modelos Two-Tower resultan útiles porque separan la representación del usuario y del producto, permitiendo comparar ambos espacios de manera eficiente durante la etapa de recomendación. Este tipo de modelo puede ser más flexible que los enfoques tradicionales, aunque también exige mayor capacidad computacional.

El reto de los sistemas de recomendación no se limita a lograr buenas métricas de predicción. En plataformas reales, también es necesario procesar grandes volúmenes de datos en tiempos razonables. Un sistema puede ser preciso, pero si tarda demasiado en generar recomendaciones, pierde utilidad práctica. Por esta razón, el rendimiento computacional es un aspecto fundamental. En escenarios de alto tráfico, como campañas masivas de ventas o eventos tipo Black Friday, el sistema debe responder rápidamente aunque aumente de forma significativa el número de usuarios e interacciones.

En este contexto, herramientas modernas de procesamiento de datos como Polars y Dask ofrecen ventajas importantes. Polars permite ejecutar operaciones tabulares de forma eficiente aprovechando múltiples núcleos de CPU y técnicas de optimización de consultas. Dask, por su parte, permite dividir el trabajo en particiones y distribuir tareas entre varios workers, lo que facilita el procesamiento de datasets grandes. Estas herramientas son relevantes para el proyecto porque permiten comparar enfoques tradicionales contra alternativas paralelas o distribuidas.

También existen herramientas orientadas a la aceleración por GPU, como RAPIDS cuDF y cuML. Estas librerías permiten ejecutar operaciones de análisis de datos y algoritmos de Machine Learning sobre GPU, lo que puede reducir considerablemente los tiempos de cómputo en tareas altamente paralelizables. En un sistema de recomendación, esta aceleración puede ser útil para procesos como clustering de usuarios, cálculo de similitudes, entrenamiento de modelos o generación de recomendaciones en tiempo reducido.

A partir de estos antecedentes, el presente proyecto se ubica en la intersección entre sistemas de recomendación, análisis de datos a gran escala y computación paralela/distribuida, utilizando como caso de estudio el dataset RetailRocket. La propuesta no se enfoca únicamente en construir un modelo de recomendación, sino también en evaluar cómo distintas estrategias computacionales afectan el rendimiento del sistema. De esta manera, el proyecto busca demostrar que la calidad de las recomendaciones y la eficiencia del procesamiento deben analizarse de forma conjunta cuando se trabaja con datos reales de comercio electrónico.

Marco teórico

Sistemas de recomendación

Los sistemas de recomendación son herramientas computacionales diseñadas para sugerir elementos relevantes a un usuario a partir de sus preferencias, historial de interacción o patrones de comportamiento. En el contexto del comercio electrónico, estos sistemas permiten recomendar productos que podrían ser de interés para cada cliente, facilitando la búsqueda dentro de catálogos extensos y mejorando la experiencia de compra.

En plataformas de e-commerce, las recomendaciones pueden generarse a partir de distintos tipos de información, como productos vistos, artículos agregados al carrito, compras realizadas o similitudes entre usuarios. Por esta razón, los sistemas de recomendación se consideran una aplicación importante del análisis de datos y del aprendizaje automático.

Retroalimentación implícita en e-commerce

En muchos sistemas de comercio electrónico no se cuenta con calificaciones directas de los usuarios. Es decir, los clientes no siempre indican explícitamente si un producto les gustó o no. En su lugar, el sistema registra acciones como visualizaciones, adiciones al carrito y compras. A este tipo de información se le conoce como retroalimentación implícita.

La retroalimentación implícita debe interpretarse cuidadosamente, ya que cada evento representa un nivel distinto de interés. Una visualización puede indicar curiosidad, una adición al carrito puede reflejar intención de compra y una transacción puede considerarse

una señal fuerte de preferencia. En este proyecto, estos eventos se utilizan como base para construir relaciones entre usuarios y productos.

Filtrado colaborativo

El filtrado colaborativo es una técnica de recomendación basada en las interacciones entre usuarios y productos. Su idea principal es que usuarios con comportamientos similares pueden compartir intereses similares. Por ejemplo, si dos usuarios han interactuado con productos parecidos, es posible recomendar a uno de ellos productos que el otro ya compró o consultó.

Este enfoque es útil cuando se dispone de una gran cantidad de datos históricos de interacción. En el caso del dataset Retail Rocket, las interacciones registradas permiten construir una matriz usuario-producto que puede ser utilizada para identificar patrones de comportamiento y generar recomendaciones personalizadas.

Factorización matricial y ALS

La factorización matricial es una técnica utilizada para representar usuarios y productos en un espacio de menor dimensión. En lugar de trabajar directamente con una matriz grande y dispersa de interacciones, el modelo aprende vectores latentes para cada usuario y cada producto. La similitud entre estos vectores permite estimar qué tan relevante puede ser un producto para un usuario.

Uno de los métodos utilizados para este tipo de problema es ALS, conocido como Alternating Least Squares. Este algoritmo ajusta de forma alternada los factores latentes de usuarios y productos. En proyectos de recomendación con retroalimentación implícita, ALS puede utilizarse como modelo base, ya que permite construir recomendaciones de manera relativamente eficiente y comparar sus resultados contra modelos más avanzados.

Modelos basados en embeddings

Los modelos basados en embeddings representan usuarios y productos mediante vectores aprendidos automáticamente durante el entrenamiento. Estos vectores buscan capturar relaciones ocultas entre los elementos del sistema. Si un usuario y un producto tienen embeddings cercanos, se interpreta que existe una posible afinidad entre ambos.

Una arquitectura útil en sistemas de recomendación es el modelo Two-Tower. En este enfoque, una parte del modelo aprende la representación del usuario y otra parte aprende la representación del producto. Luego, ambas representaciones se comparan para estimar la probabilidad de interacción. Este diseño puede ser conveniente para sistemas de recomendación a gran escala, ya que permite calcular previamente representaciones de productos y acelerar la inferencia.

Computación paralela y distribuida

La computación paralela consiste en dividir una tarea en varias partes que pueden ejecutarse simultáneamente. Esto permite aprovechar múltiples núcleos de CPU o unidades de procesamiento para reducir el tiempo total de ejecución. En este proyecto, el paralelismo resulta importante porque el dataset contiene millones de eventos de interacción, lo que puede volver lento el procesamiento secuencial tradicional.

La computación distribuida, por su parte, permite repartir el trabajo entre varios procesos, workers o nodos. Esta estrategia es útil cuando el volumen de datos crece o cuando se busca simular escenarios de alta demanda, como picos de tráfico en eventos comerciales masivos. En el sistema propuesto, el procesamiento distribuido se considera principalmente para particionar datos, ejecutar transformaciones y evaluar la escalabilidad del pipeline.

Aceleración por GPU

La aceleración por GPU se utiliza cuando una tarea puede dividirse en muchas operaciones pequeñas que se ejecutan en paralelo. A diferencia de una CPU, que posee pocos núcleos más complejos, una GPU cuenta con una gran cantidad de unidades de procesamiento diseñadas para ejecutar operaciones masivamente paralelas.

En este proyecto, la GPU puede utilizarse en tareas como clustering de usuarios, cálculo de similitudes y entrenamiento de modelos basados en embeddings. Herramientas como RAPIDS cuML y PyTorch permiten aprovechar este tipo de hardware para reducir tiempos de entrenamiento o análisis en comparación con enfoques ejecutados únicamente en CPU.

Herramientas de procesamiento de datos

Polars es una biblioteca para procesamiento de datos tabulares que aprovecha el paralelismo multinúcleo y optimización de consultas. Su uso se justifica en etapas como limpieza,

filtrado, agregaciones y joins, donde se requiere procesar grandes volúmenes de datos de forma eficiente.

Dask permite trabajar con datos divididos en particiones y distribuir tareas entre varios workers. Esta herramienta es útil para simular el procesamiento distribuido y evaluar la escalabilidad del pipeline. Por otro lado, PyArrow y el formato Parquet permiten almacenar datos en formato columnar, facilitando la interoperabilidad entre distintas herramientas y reduciendo costos de entrada/salida.

Métricas de calidad del sistema de recomendación

La evaluación de un sistema de recomendación requiere métricas orientadas al ranking. No basta con saber si el sistema acierta o falla, sino que también importa el orden en que aparecen los productos recomendados.

Entre las métricas relevantes se encuentran Hit Rate@K, MAP@K y NDCG@K. Hit Rate@K indica si al menos un producto relevante aparece dentro de las primeras K recomendaciones. MAP@K evalúa la precisión promedio considerando el orden de los elementos recomendados. NDCG@K asigna mayor importancia a los aciertos ubicados en las primeras posiciones del ranking.

Métricas de rendimiento computacional

Debido a que el proyecto pertenece al área de computación paralela y distribuida, también es necesario evaluar el rendimiento computacional. Las métricas principales son tiempo de ejecución, speedup y eficiencia.

El tiempo de ejecución mide cuánto tarda una etapa del pipeline en completarse. El speedup compara el tiempo de una versión secuencial contra una versión paralela o distribuida. La eficiencia relaciona el speedup obtenido con la cantidad de recursos utilizados, permitiendo analizar si el uso de más núcleos, workers o GPU realmente produce una mejora proporcional.

Estas métricas permiten justificar técnicamente el uso de herramientas paralelas y distribuidas, demostrando si la solución propuesta mejora el procesamiento frente a un enfoque tradicional.

Metodología

La metodología del proyecto se desarrollará mediante un flujo de trabajo dividido en etapas, con el fin de construir, evaluar e integrar un sistema de recomendación para comercio electrónico basado en procesamiento paralelo y distribuido. Cada etapa cumple una función específica dentro del pipeline general, desde la adquisición de datos hasta la visualización de resultados y la evaluación del rendimiento computacional.

1. Adquisición y revisión inicial de datos

La primera etapa consiste en utilizar el dataset RetailRocket E-commerce Dataset, el cual contiene aproximadamente 2.7 millones de eventos de interacción entre usuarios y productos. Estos eventos incluyen visualizaciones, adiciones al carrito y transacciones, además de información complementaria sobre categorías y propiedades de productos.

En esta fase se revisará la estructura de los archivos disponibles, los tipos de datos, la cantidad de registros, la relación entre las tablas y la calidad general de la información. También se identificarán posibles valores faltantes, duplicados o inconsistencias que deban ser tratados antes del análisis y modelado.

2. Preprocesamiento de datos

Después de la revisión inicial, se realizará la limpieza y transformación del conjunto de datos. Esta etapa incluirá la normalización de tipos de datos, eliminación de registros inconsistentes, integración de fuentes relacionadas y creación de variables derivadas a partir de las interacciones usuario-producto.

Para mejorar el rendimiento del procesamiento, se utilizarán herramientas como Polars y Dask. Polars se emplea para operaciones tabulares eficientes, como filtrados, agregaciones y joins. Dask se utilizará en aquellas tareas donde sea necesario particionar el dataset y distribuir el procesamiento entre varios workers. La salida del preprocesamiento se almacenará en formato Parquet, con el fin de facilitar su reutilización en las siguientes etapas.

3. Análisis exploratorio de datos

Con los datos limpios, se realizará un análisis exploratorio orientado a comprender el comportamiento de los usuarios y productos dentro del sistema. Se estudiará la distribución de eventos, la frecuencia de interacción por usuario, la popularidad de productos y la proporción entre visualizaciones, adiciones al carrito y compras.

También se analizarán patrones relevantes que puedan apoyar el diseño del sistema de recomendación, como usuarios con alto nivel de actividad, productos con mayor frecuencia de interacción y posibles diferencias entre tipos de eventos. Los resultados de esta etapa permitirán seleccionar variables útiles para el modelado posterior.

4. Segmentación de usuarios

Como parte del análisis del comportamiento, se aplicarán técnicas de clustering para identificar grupos de usuarios con patrones similares de interacción. Para ello se construirán variables agregadas por usuario, tales como cantidad total de eventos, diversidad de productos consultados, número de compras y frecuencia de actividad.

Se evaluará una implementación en CPU mediante Scikit-Learn y, si el entorno de ejecución lo permite, una versión acelerada por GPU mediante RAPIDS cuML. Esta comparación permitirá observar si la aceleración por GPU aporta una mejora significativa en tiempo de ejecución para esta etapa.

5. Construcción del sistema de recomendación

El sistema de recomendación se desarrollará en dos niveles. Primero se implementará un modelo base de filtrado colaborativo, que servirá como referencia inicial para medir la calidad de las recomendaciones. Posteriormente, se evaluará un modelo más avanzado basado en embeddings, implementado con PyTorch, con posibilidad de utilizar aceleración por GPU durante el entrenamiento.

Ambos modelos utilizarán las interacciones históricas entre usuarios y productos para generar recomendaciones personalizadas. La salida esperada será una lista top-N de productos recomendados para cada usuario.

6. Evaluación de recomendaciones

La calidad del sistema será evaluada mediante métricas orientadas al ranking, tales como Hit Rate@K, MAP@K y NDCG@K. Estas métricas permitirán analizar si los productos relevantes aparecen dentro de las primeras posiciones recomendadas y si el orden de las recomendaciones es adecuado.

Además, se revisarán casos donde el sistema presente menor desempeño, especialmente usuarios con pocas interacciones o productos con baja frecuencia. Este análisis permitirá identificar limitaciones del modelo y posibles mejoras para futuras iteraciones.

7. Evaluación del rendimiento computacional

De forma transversal, se medirán los tiempos de ejecución de las principales etapas del pipeline: carga de datos, limpieza, transformaciones, agregaciones, clustering, entrenamiento e inferencia. Se compararon versiones base contra versiones paralelas o distribuidas cuando sea posible.

Las métricas consideradas serán tiempo de ejecución, speedup, eficiencia, uso de memoria y latencia de inferencia. También se utilizarán herramientas de profiling para identificar cuellos de botella y determinar qué partes del sistema se benefician más del paralelismo multinúcleo, procesamiento distribuido o aceleración por GPU.

8. Visualización e integración final

Finalmente, se desarrollará un dashboard interactivo utilizando herramientas como Dash y Plotly. Este dashboard permitirá visualizar estadísticas del dataset, segmentos de usuarios, recomendaciones generadas, métricas de calidad del modelo y resultados de rendimiento.

La integración final del proyecto se organizará en un repositorio GitLab, incluyendo código fuente, notebooks, resultados experimentales, instrucciones de ejecución y documentación necesaria para reproducir el trabajo.

Problema

Contexto

El comercio electrónico genera diariamente millones de interacciones entre usuarios y productos (visualizaciones, adiciones al carrito, compras). Estos datos son la materia prima de los sistemas de recomendación, componentes que hoy resultan indispensables para mejorar la experiencia del cliente, aumentar la conversión y fortalecer la fidelización.

Problema central

El reto no es solo generar recomendaciones precisas, sino generarlas **rápido y a gran escala**. Los enfoques secuenciales tradicionales no logran procesar el volumen de datos que producen plataformas reales en tiempos aceptables, y esta limitación se agrava drásticamente durante picos de tráfico (como un Black Friday), donde el número de interacciones puede multiplicarse en cuestión de minutos. Un sistema que funciona bien en condiciones normales puede volverse inutilizable justo cuando más se necesita: en el momento de mayor demanda del negocio.

Esto plantea una pregunta concreta: **¿cómo diseñar un sistema de recomendación que mantenga baja latencia y alta calidad de predicción incluso cuando el volumen de datos y usuarios crece de forma masiva y repentina?**

Motivación

Responder a esta pregunta exige ir más allá del algoritmo de recomendación en sí y abordar el problema desde la arquitectura de cómputo: ¿qué tareas conviene paralelizar en CPU multinúcleo?, ¿cuáles se benefician de la aceleración GPU?, ¿cuándo es necesario distribuir la carga entre varios nodos? Este proyecto usa el dataset RetailRocket (~2.7 millones de eventos) como caso de estudio realista para explorar estas decisiones, permitiendo al equipo evidenciar de forma práctica y medible mediante métricas como speedup y eficiencia cómo la computación paralela y distribuida resuelve un problema que afecta directamente a millones de usuarios en su vida cotidiana.

Objetivos Específicos

1. Analizar y preparar los datos de interacción usuario-producto, mediante el estudio estadístico de su distribución, la identificación de correlaciones relevantes en el comportamiento de los usuarios y la selección de las variables más representativas para el modelo de recomendación, aprovechando procesamiento paralelo para acelerar esta etapa sobre el volumen completo del dataset.
2. Diseñar e implementar un pipeline distribuido de recomendación, que incluya la segmentación de usuarios mediante algoritmos de clustering y el cálculo de similitud/predicción acelerado por GPU, garantizando que el sistema pueda generar recomendaciones en tiempo real a partir de grandes volúmenes de datos.
3. Evaluar el rendimiento y la escalabilidad del sistema en escenarios que simulan condiciones reales de comercio electrónico (incluyendo picos de tráfico tipo Black Friday), midiendo métricas como speedup, eficiencia y latencia para comparar el impacto de las distintas estrategias de paralelización (CPU, GPU y clústeres) frente a un enfoque secuencial.

Justificación técnica (de herramientas o librerías a utilizar)

Las herramientas del proyecto se seleccionaron bajo un criterio único: cada una debe demostrar de forma medible un tipo de paralelismo (multinúcleo, GPU o distribuido) sobre el dataset RetailRocket (~2.7 millones de eventos), de manera que el beneficio de paralelizar sea cuantificable con métricas de speedup y eficiencia. Además, se priorizaron librerías con una ruta de migración clara entre CPU y GPU, dado que el equipo trabajará en máquinas heterogéneas (algunos integrantes sin GPU local) y necesita que el pipeline sea reproducible en ambos escenarios.

Ingesta y preprocesamiento de datos

Se descartó Pandas como herramienta principal porque opera en un único hilo y carga el dataset completo en memoria, lo cual es exactamente el cuello de botella que buscamos eliminar. En su lugar se seleccionó **Polars** como motor principal de limpieza y transformación, y **Dask** para las operaciones que requieren particionar el dataset entre varios workers.

Herramienta	Paralelismo	Evaluación	GPU	Proyecto
Pandas	1 hilo	Eager (inmediata)	no	Descartada referencia de baseline secuencial
Polars	Multinúcleo (Rust, SIMD)	Lazy (plan de ejecución optimizado)	no	Limpieza, joins, agregaciones sobre eventos.csv
Dask	Distribuido (local cluster / multi-nodo)	Lazy (grafo de tareas)	no	Particionado del dataset y escalado horizontal simulado
cuDF	GPU	Eager	si	Prueba de aceleración GPU sobre las transformaciones más costosas

Polars se justifica frente a Dask para las transformaciones tabulares simples porque su motor en Rust con SIMD ya explota el paralelismo multinúcleo sin la sobrecarga de un scheduler distribuido; Dask se reserva para las etapas donde el dataset se particiona explícitamente (por categoría o por fecha) y se necesita simular procesamiento distribuido real. La salida de esta etapa se estandariza en Parquet particionado (vía PyArrow), formato columnar que reduce el I/O en las etapas posteriores y es el estándar de interoperabilidad entre Polars, Dask y cuDF.

Análisis exploratorio y clustering de usuarios

Para el EDA y la segmentación de usuarios se comparan Scikit-Learn (CPU, **n_jobs=-1**) y RAPIDS cuML (GPU):

Criterio	Scikit-Learn	RAPIDS cuML
Paralelismo	Multinúcleo vía joblib	Miles de núcleos CUDA
Curva de aprendizaje	Baja (API conocida)	Baja misma API que Scikit-Learn

Ventaja principal	No requiere hardware especial, ideal para prototipar	Speedup significativo en K-Means/DBSCAN sobre millones de filas
Uso en el proyecto	Baseline de clustering y métricas descriptivas	Comparativa de speedup CPU vs GPU

Se eligió esta combinación en lugar de usar solo una de las dos porque el Objetivo específico 1 exige explícitamente medir el efecto de paralelizar el EDA: sin un baseline CPU no hay speedup que reportar.

Modelado y aceleración por GPU

El núcleo del sistema de recomendación combina un modelo baseline y un modelo acelerado por GPU, siguiendo el Objetivo 2:

- Implicit (ALS): filtrado colaborativo multi-hilo sobre CPU. Se usa como baseline porque es rápido de entrenar, no requiere GPU y da una cota inferior de calidad (NDCG@K, MAP@K) contra la cual comparar.
- PyTorch: para un modelo de embeddings/Two-Tower entrenado con aceleración GPU. Se prefiere sobre JAX por tener mejor soporte de despliegue (TorchScript/ONNX) y curva de aprendizaje más suave para el equipo.
- RAPIDS cuML: se evalúa como alternativa para el cálculo de similitud (vecinos más cercanos) acelerado por GPU, requerido para las recomendaciones en tiempo real.

La estrategia acordada es desarrollar y depurar el código con las versiones CPU (Polars/Scikit-Learn/Implicit) y ejecutar las corridas GPU en Google Colab o Kaggle Notebooks (que ofrecen GPU gratuita), documentando el entorno exacto para reproducibilidad.

Simulación de picos de tráfico (Black Friday) y escalabilidad

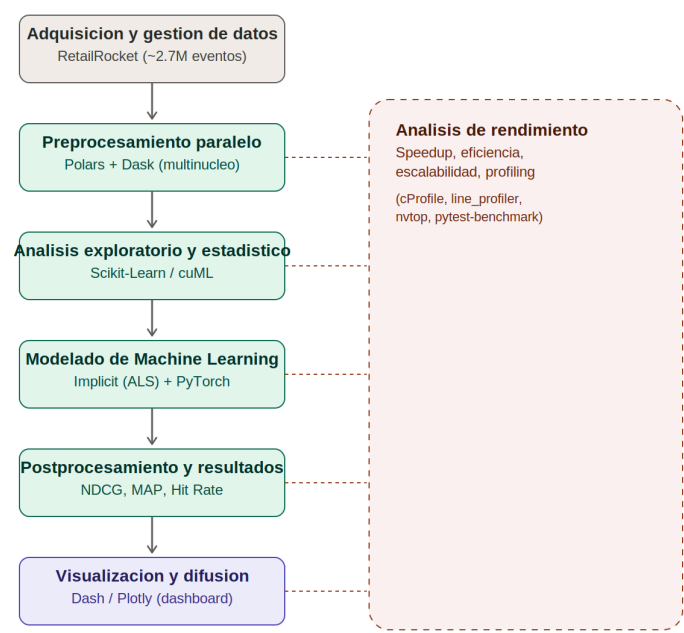
Para el Objetivo 3 (evaluación de rendimiento ante picos de tráfico), Dask se reutiliza para simular el aumento del volumen de eventos concurrentes (escalabilidad débil) y para repartir la carga entre varios workers locales (escalabilidad fuerte). El profiling de cada etapa se hace con cProfile y line_profiler, por ser herramientas estándar de Python que no añaden dependencias pesadas y permiten identificar cuellos de botella a nivel de función.

Diseño de la arquitectura general del sistema

1. Adquisición y gestión de datos. El pipeline inicia con la ingesta del dataset RetailRocket E-commerce Dataset (~2.7 millones de eventos de interacción usuario-producto, más los catálogos de categorías y propiedades de producto). Esta etapa no aplica todavía ninguna técnica de paralelismo: su función es dejar los datos crudos disponibles para la siguiente etapa.
2. Preprocesamiento paralelo. Los datos crudos se limpian, integran y transforman utilizando Polars (paralelismo multinúcleo vía su motor en Rust) y Dask (procesamiento distribuido por particiones). La salida de esta etapa se estandariza en formato Parquet particionado, que sirve de entrada común para las siguientes etapas.
3. Análisis exploratorio y estadístico. Sobre los datos ya preprocesados se ejecuta el análisis descriptivo, la identificación de correlaciones y la segmentación de usuarios mediante clustering, usando Scikit-Learn como base y RAPIDS cuML como alternativa acelerada por GPU para comparar el impacto de paralelizar esta etapa.
4. Modelado de Machine Learning. Con las variables seleccionadas en la etapa anterior, se entrena primero un modelo baseline de filtrado colaborativo (ALS, vía la librería Implicit, paralelizado en CPU) y luego un modelo más avanzado basado en PyTorch, que puede beneficiarse de aceleración GPU para el entrenamiento.
5. Postprocesamiento y análisis de resultados. Los resultados de ambos modelos se evalúan con métricas estándar de sistemas de recomendación (NDCG@K, MAP@K, Hit Rate@K), y se realiza un análisis de errores para identificar en qué casos el modelo falla.
6. Visualización y difusión de resultados. Finalmente, un dashboard interactivo construido con Dash/Plotly expone las recomendaciones generadas, los resultados del EDA y las métricas de rendimiento, permitiendo explorar el sistema de forma interactiva.

Módulo transversal: análisis de rendimiento. A diferencia de las seis etapas anteriores, este módulo no ocupa un lugar fijo en la secuencia: mide el tiempo de ejecución, el speedup y la eficiencia de las etapas 2 a 6, comparando siempre una versión secuencial (1 worker) contra versiones paralelas (2, 4, 8 workers), e identifica los cuellos de botella de cada una usando cProfile, line_profiler y, cuando aplique, nvidia-top para monitoreo de GPU. Este análisis

transversal es lo que permite justificar, con evidencia y no solo con intención, por qué el proyecto usa procesamiento paralelo y distribuido.



Plan de trabajo

Entrega 1 — Fundamentos y Diseño del Proyecto (20%)

Semana	Actividad	Responsable(s)
Semana 7	Definición del problema, revisión de literatura y selección final del dataset (RetailRocket E-commerce Dataset).	Todo el equipo
Semana 7	Redacción de la introducción, el planteamiento del problema y los objetivos específicos.	2 integrantes
Semana 7	Investigación y justificación técnica de herramientas a utilizar (Dask, PyArrow, cuDF/cuML, etc.).	2 integrantes
Semana 7	Definición de la arquitectura general del pipeline computacional (diagrama de flujo).	1 integrante (líder técnico)

Semana 7	Elaboración del cronograma y plan de trabajo del proyecto.	1 integrante
Semana 7	Consolidación del documento, formato IEEE y revisión final.	Todo el equipo

Entrega 2 — Implementación Inicial y Validación Parcial (30%)

Semana	Actividad	Responsable(s)
Semana 8	Ingesta de datos y preprocesamiento paralelo/distribuido: limpieza, integración y agregaciones con Dask/Polars.	2 integrantes
Semanas 8–9	Análisis exploratorio y estadístico: distribución de interacciones usuario-producto y correlaciones relevantes.	2 integrantes
Semana 9	Clustering de usuarios y selección de variables representativas para el modelo.	1 integrante
Semana 9	Primeras pruebas de aceleración por GPU para cálculos de similitud y predicción.	1–2 integrantes
Semana 9	Validación preliminar de resultados y pruebas unitarias del pipeline.	Todo el equipo
Semana 10	Documentación de obstáculos encontrados y redacción del informe de avance.	Todo el equipo

Entrega 3 — Proyecto Final Completo (40%)

Semana	Actividad	Responsable(s)
Semana 11	Modelado de Machine Learning del sistema de recomendación (Scikit-Learn / cuML / PyTorch).	2 integrantes
Semanas 11	Escalamiento distribuido del modelo y simulación de picos de tráfico (escenario Black Friday).	2 integrantes
Semana 11	Postprocesamiento: métricas de desempeño, análisis de errores e interpretabilidad.	1–2 integrantes
Semana 11	Evaluación de rendimiento: speedup, eficiencia, escalabilidad, uso de GPU y memoria.	2 integrantes
Semana 11	Desarrollo del dashboard interactivo (Dash / Plotly / Panel).	1–2 integrantes
Semana 11	Redacción del informe final completo (IEEE), conclusiones y lecciones aprendidas.	Todo el equipo
Semana 11	Revisión final de código, README y verificación de reproducibilidad en GitLab.	Todo el equipo

Entrega 4 — Presentación (10%)

Semana	Actividad	Responsable(s)
Semana 12	Preparación de la presentación y demostración en vivo del dashboard.	Todo el equipo
Semana 12	Ensayo general y distribución de tiempos de exposición entre integrantes.	Todo el equipo
Semana 12	Presentación final del proyecto.	Todo el equipo

Referencias:

Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. 2008 Eighth IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 263–272.

X. Yi, J. Yang, L. Hong, D. Z. Cheng, L. Heldt, A. Kumthekar, Z. Zhao, L. Wei, and E. Chi, "Sampling-bias-corrected neural modeling for large corpus item recommendations," in *Proc. 13th ACM Conf. Recommender Systems (RecSys '19)*, 2019, pp. 269–277.

Polars, "Polars documentation," 2024. [Online]. Available: <https://docs.pola.rs>

Dask Development Team, "Dask: Library for dynamic task scheduling," 2016. [Online]. Available: <https://dask.org>

RAPIDS Development Team, "RAPIDS: Libraries for end-to-end GPU data science," NVIDIA, 2023. [Online]. Available: <https://rapids.ai>

Retailrocket, "Retailrocket recommender system dataset," Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>

Zhou, Wilkinson, Schreiber & Pan (2008) *"Large-scale Parallel Collaborative Filtering for the Netflix Prize"* (AAIM 2008, LNCS vol. 5034, pp. 337–348)

He, Liao, Zhang, Nie, Hu & Chua (2017) *"Neural Collaborative Filtering"* (WWW '17, pp. 173–182).

Amdahl (1967) *"Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities"* (AFIPS Spring Joint Computer Conf., pp. 483–485)

Järvelin & Kekäläinen (2002) *"Cumulated gain-based evaluation of IR techniques"* (ACM TOIS, 20(4), 422–446).

Herlocker, Konstan, Terveen & Riedl (2004) *"Evaluating Collaborative Filtering Recommender Systems"* (ACM TOIS, 22(1), 5–53).